

DATAQX

STATELESS INTELLIGENT DATA QUALITY, CLEANING & POWER BI PREPARATION PLATFORM

You are acting as a:

Senior Full-Stack Engineer + Senior Data Engineer + Senior Data Analyst + Data Quality Engineer + BI Engineer + QA Engineer + UI/UX Engineer + Python Architect.

Your task is to build a **professional, production-style, database-free, stateless web application** called:

DataQX

Intelligent Data Quality & Preparation Platform

DataQX is a dataset-processing platform that accepts messy datasets, intelligently analyzes them, cleans them according to project requirements, validates the results, prepares them for analytics/Power BI, and generates a complete downloadable output package.

This is NOT a simple Python cleaning script.

This is a complete web-based application consisting of:

- Modern frontend
- FastAPI backend
- Real data-processing engine
- Multi-format ingestion
- Project-aware cleaning
- Dataset profiling
- Automatic issue detection
- Confidence-based cleaning
- Data lineage
- Audit logging
- Data drift detection
- Validation gates
- Rollback
- Power BI readiness analysis
- Data dictionary

- Quality scoring
 - Performance monitoring
 - Professional PDF reporting
 - Downloadable results
-

0. ABSOLUTE ARCHITECTURE RULES

RULE 1 — NO DATABASE

DataQX must NOT use any database.

Absolutely do NOT use:

PostgreSQL

MySQL

MongoDB

SQLite

Supabase Database

Neon Database

Firebase Database

Redis as persistent storage

Any SQL database

Any NoSQL database

Any cloud database

Do not introduce a database as an alternative.

RULE 2 — NO ORM

Do NOT use:

SQLAlchemy

Prisma

Django ORM

MongoEngine

Tortoise ORM

Any ORM

There must be no ORM layer.

RULE 3 — NO DATABASE INFRASTRUCTURE

Do NOT create:

Database schemas

Database tables

Database migrations

Database connections

Database CRUD APIs

Database models

Database repositories

Database authentication

Database-backed sessions

RULE 4 — NO DATABASE-BACKED HISTORY

Do NOT store:

Run history

Audit logs

Cleaning logs

Lineage

Drift history

Datasets

Reports

User projects

in a database.

🚀 1. STATELESS ARCHITECTURE

DataQX must operate as a **stateless dataset-processing platform**.

The primary workflow is:

Upload Dataset



Analyze



Profile



Detect Issues



Recommend Cleaning



Apply Cleaning



Validate



Generate Reports



Download Results

The backend processes the dataset during the run.

Generated artifacts are written to temporary/local filesystem locations.

The application does NOT depend on persistent application state.

2. PRIMARY USER EXPERIENCE

The user should be able to:

1. Open DataQX
2. Upload a dataset

3. Upload or enter a project plan
 4. Click Analyze
 5. Review dataset profile
 6. Review detected issues
 7. Review cleaning recommendations
 8. Apply safe cleaning
 9. Review validation
 10. Review Power BI readiness
 11. View lineage
 12. View drift if historical profile files are available
 13. Download the complete output package
- The user should NOT need to write Python.
-

3. TECHNOLOGY STACK

Frontend

Use:

React

Vite

TypeScript where practical

Tailwind CSS

Recharts

Backend

Use:

Python

FastAPI

Pandas

NumPy

PyArrow

OpenPyXL

PyYAML

ReportLab

SciPy where useful

Scikit-learn only where genuinely useful

Testing

Use:

pytest

Frontend testing:

Vitest

or another lightweight appropriate framework.

4. NO DOCKER

Do NOT use:

Docker

Docker Compose

Kubernetes

Containers

Container orchestration

The project must run directly on a normal Windows development machine.

Local development should require only:

Python

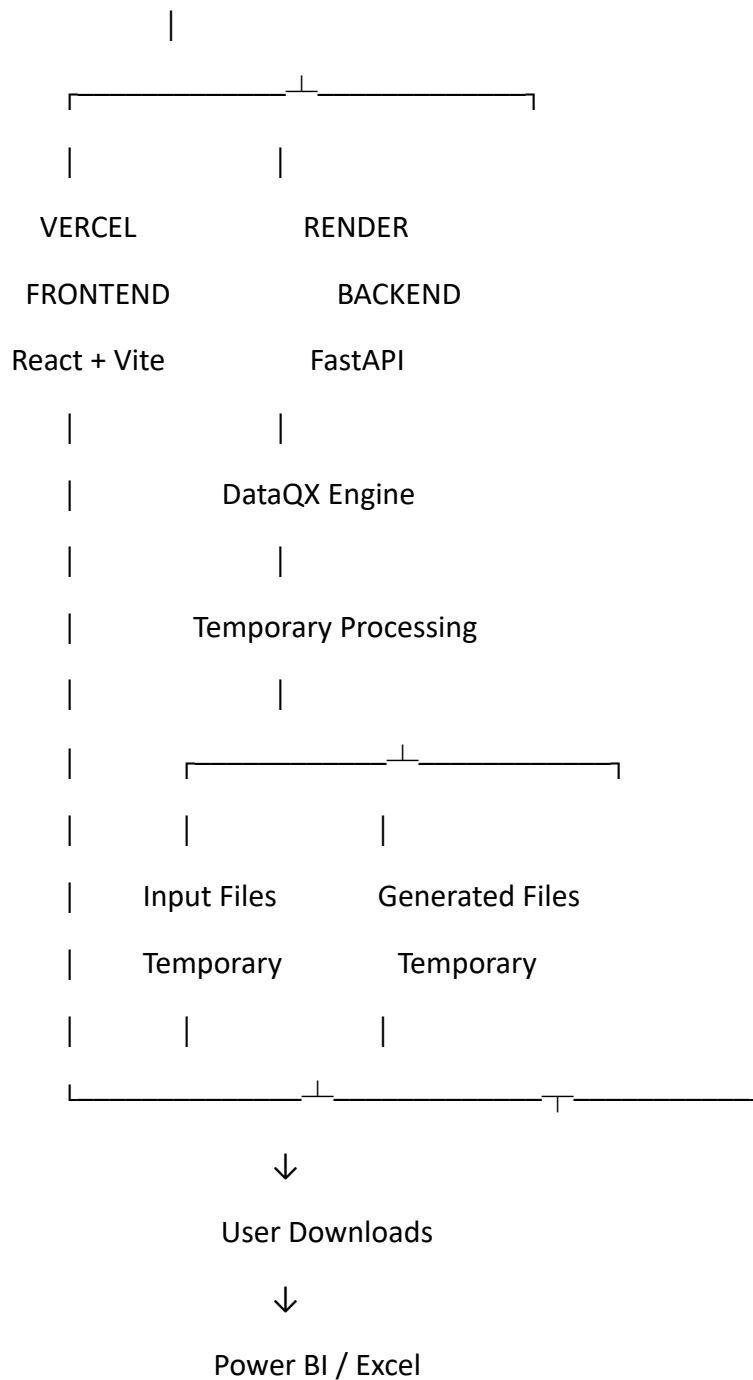
Node.js

npm

5. DEPLOYMENT ARCHITECTURE

The intended architecture is:

DATAQX



IMPORTANT

There is:

NO DATABASE

The backend must not depend on persistent application storage.

Generated files are processing artifacts.

Do not add a database later.

6. DEPLOYMENT STORAGE RULE

The application must be designed around temporary/ephemeral processing storage.

Do not assume that backend filesystem storage is permanent on cloud deployment.

For cloud deployment:

Upload



Temporary processing



Generate artifacts



User downloads



Temporary artifacts can expire

The current version does NOT require permanent cloud storage.

Do not introduce object storage merely to compensate for the absence of a database unless explicitly requested later.

7. PROJECT DIRECTORY

Create:

dataqx/

|

|— backend/

| |— main.py

| |— requirements.txt

| |

| |— app/

| | |— api/

| | |— core/


```
| | └─ models/
| | └─ services/
| | └─ utils/
| |
| └─ tests/
|
└─ frontend/
| └─ package.json
| └─ src/
| | └─ components/
| | └─ pages/
| | └─ services/
| | └─ hooks/
| | └─ types/
| | └─ utils/
| └─ tests/
|
└─ data/
| └─ input/
| └─ output/
| └─ temp/
| └─ samples/
|
└─ project/
| └─ project_plan.md
|
└─ reports/
```

```
|  └─ runs/
|  └─ history/
|
|  └─ logs/
|
|  └─ config/
|
|  └─ README.md
└─ .gitignore
```

8. FILE-BASED STORAGE ONLY

Generated artifacts must use filesystem storage.

Example:

```
data/
|  └─ input/
|  └─ output/
|  └─ temp/
└─ samples/
```

```
reports/
|  └─ runs/
└─ history/
```

```
logs/
```

No database-backed storage.

9. OUTPUT ARTIFACTS

Generate files such as:

cleaned_dataset.csv

cleaned_dataset.xlsx

data_dictionary.csv

cleaning_log.csv

audit_log.csv

data_lineage.csv

data_lineage.json

drift_report.csv

validation_report.json

quality_report.json

before_after_summary.json

performance_log.csv

DataQX_Report.pdf

Only generate formats that are relevant to the input/project, but the architecture should support them.

10. ORIGINAL DATA PROTECTION

The original dataset must remain untouched.

Raw files:

data/input/

must never be overwritten.

Clean files:

data/output/

Temporary processing:

data/temp/

11. PROJECT PLAN

Allow the user to provide:

project/project_plan.md

or enter requirements through the frontend.

The project plan can contain:

Project objective

Business problem

Business questions

KPIs

Important columns

Required columns

Expected calculations

Expected dashboard

Expected visualizations

Filters

Time period

Target variables

Business rules

Columns that must not be modified

Power BI requirements

Expected output

12. PROJECT-AWARE CLEANING

Project requirements have priority.

Use:

1. Explicit project requirements
2. Domain/business rules
3. Dataset evidence
4. Statistical evidence
5. Generic cleaning rules

The same dataset may receive different recommendations depending on the project.

13. SUPPORTED INPUT FORMATS

Support at minimum:

CSV

TSV

TXT

XLSX

XLS

JSON

PARQUET

Where practical:

XML

FEATHER

Automatically detect file type.

Never assume CSV.

14. MULTIPLE FILE HANDLING

If multiple files are uploaded:

customers.csv

orders.csv

products.csv

determine whether they are:

Independent

Related

Different versions

Duplicate copies

Detect relationships.

Do not blindly merge unrelated datasets.

15. DATASET PROFILING

Before cleaning, create a baseline profile.

Dataset-level

Calculate:

Rows

Columns

File size

Memory usage

Duplicate rows

Empty rows

Empty columns

Constant columns

Near-constant columns

Dataset hash

Column-level

Calculate:

Original name

Clean name

Data type

Inferred type

Missing count

Missing percentage

Unique count

Unique percentage

Min

Max

Mean

Median

Standard deviation

Mode

Zero count

Negative count

Example values

Categorical:

Top categories

Rare categories

Frequency

Potential inconsistencies

Numerical:

Quantiles

Distribution

Outliers

Dates:

Minimum

Maximum

Invalid dates

Future dates

Format patterns

Text:

Whitespace

Empty strings

Encoding

Case variation

Unicode issues

16. ISSUE DETECTION

Detect relevant issues including:

Missing values

Exact duplicates

Duplicate IDs

Potential near duplicates

Mixed data types

Invalid data types

Formatting inconsistencies

Whitespace

Category inconsistencies

Invalid dates

Ambiguous dates

Impossible values

Negative values

Outliers

Encoding problems

Corrupted characters

Invalid IDs

Referential integrity problems

Business-rule violations

Schema problems

Constant columns

Potentially irrelevant columns

High-cardinality columns

17. MISSING VALUE DETECTION

Detect:

NaN

NULL

None

N/A

NA

n/a

null

-

--

?

blank

empty

whitespace

Unknown

Not Available

Do not automatically classify every textual placeholder as missing.

Use contextual evidence.

18. MISSING VALUE HANDLING

Never blindly use:

0

mean

median

mode

Choose based on:

Data type

Missing percentage

Distribution

Project requirements

Business meaning

Related columns

Group behavior

Possible strategies:

Leave missing

Drop row

Drop column

Mean

Median

Mode

Forward fill

Backward fill

Interpolation

Group-wise imputation

Unknown

Not Available

Every modification must be logged.

19. DUPLICATE DETECTION

Detect:

Exact duplicates

Duplicate primary/unique keys

Potential near duplicates

Do not blindly merge fuzzy matches.

20. COLUMN NAME NORMALIZATION

Detect:

Customer Name

customer-name

CUSTOMER NAME

customer.name

Potential output:

customer_name

Maintain the original-to-clean mapping.

21. DATA TYPE INFERENCE

Infer:

Integer

Float

Decimal

Boolean

Date

Datetime

Categorical

String

ID

Protect identifiers.

Example:

001234

must remain:

001234

when leading zeros are meaningful.

22. NUMERIC CLEANING

Handle:

₹50,000

\$1,200

1,000

"500"

500.00

5%

Detect:

Currency

Percentages

Thousands separators

Invalid numeric strings

Mixed types

Do not blindly convert IDs.

23. TEXT CLEANING

Handle:

Leading spaces

Trailing spaces

Multiple spaces

Newlines

Case inconsistencies

Unicode normalization

Encoding issues

24. CATEGORY STANDARDIZATION

Detect:

Male

male

MALE

M

and:

USA

US

U.S.A.

United States

Use confidence-based standardization.

Low-confidence mappings must be flagged.

25. DATE VALIDATION

Support common formats:

YYYY-MM-DD

DD/MM/YYYY

MM/DD/YYYY

DD-MMM-YYYY

Use project and dataset context.

For ambiguous dates:

DO NOT GUESS.

FLAG FOR REVIEW.

Detect:

Invalid dates

Impossible dates

Future dates

Inconsistent formats

Timezone issues

Out-of-scope dates

26. OUTLIER DETECTION

Use:

IQR

Z-score

Modified Z-score

Percentiles

Domain-specific limits

Do not automatically delete outliers.

Classify:

Legitimate

Suspicious

Potential error

Invalid

27. IMPOSSIBLE VALUES

Detect values such as:

Age < 0

Age > 120

Percentage < 0

Percentage > 100

Rating < 0

Rating > 5

Quantity < 0

But consider domain context.

For example, negative revenue could represent:

Refund

Return

Chargeback

28. BUSINESS RULE ENGINE

Support relationships such as:

Revenue $\approx$ Quantity $\times$ Unit Price

End Date $\geq$ Start Date

Total = Subtotal + Tax

Age consistent with Date of Birth

Allow project-specific business rules.

29. ID VALIDATION

Check:

Missing IDs

Duplicate IDs

Invalid format

Invalid length

Unexpected characters

Leading-zero loss

Never modify identifiers without strong evidence.

30. REFERENTIAL INTEGRITY

For multiple datasets:

orders.customer_id

↓

customers.customer_id

Validate:

Primary keys

Foreign keys

Orphan records

Duplicate dimension keys

Missing relationships

Do not automatically delete orphan records.

31. CONFIDENCE FRAMEWORK

Every transformation must have:

HIGH

MEDIUM

LOW

HIGH

Safe to automatically apply.

Examples:

Trim whitespace

Remove UTF-8 BOM

Safe formatting normalization

MEDIUM

Apply with strong evidence and log it.

Examples:

Category standardization

Date normalization

Certain numeric conversions

LOW

Do not automatically modify.

Flag for human review.

Examples:

Ambiguous dates

Fuzzy category matches

Potential semantic changes

Suspicious outliers

Every action must record:

Rule

Confidence

Evidence

Action

Reason

32. DATA LINEAGE

Track:

RAW COLUMN



TRANSFORMATION



CLEAN COLUMN

For every output column:

lineage_id

source_dataset

source_column

output_column

transformation

rule

reason

confidence

For derived columns:

revenue = quantity × unit_price

record both source columns and the transformation.

Generate:

data_lineage.csv

data_lineage.json

The frontend must visualize lineage using data returned from FastAPI.

No database is required.

33. AUDIT LOGGING

Audit logging must remain fully functional without a database.

Generate:

logs/audit_log.csv

logs/cleaning_log.csv

Every modification should record:

timestamp

run_id

dataset

column

row_reference

issue_type

original_value

new_value

action

rule

reason

confidence

severity

status

For very large datasets, intelligently aggregate repetitive changes rather than generating millions of unnecessary records.

34. FILE-BASED RUN METADATA

There is NO persistent database-backed run history.

If run metadata is needed, store it as files.

Example:

reports/

└─ runs/

└─ run_2026_001/

└─ run_metadata.json

└─ quality_report.json

└─ cleaning_log.csv

└─ validation_report.json

└─ data_lineage.csv

A run directory may contain all artifacts generated during that run.

The run metadata should include:

run_id

timestamp

dataset name

input hash

output hash

project name

processing status

quality score

Power BI readiness

processing time

This is FILE-BASED ONLY.

35. DATA DRIFT DETECTION

Drift detection must work without a database.

Use previously generated profile/report files.

Example:

reports/

└─ history/

 └─ profile_run_001.json

 └─ profile_run_002.json

└─ profile_run_003.json

Compare current profile against previous saved profiles.

Detect:

Schema drift

New columns

Removed columns

Renamed columns

Changed data types

Volume drift

Row count changes

Missingness drift

Example:

Previous: 2%

Current: 17%

Category drift

Detect:

New categories

Removed categories

Frequency changes

Distribution drift

Compare:

Mean

Median

Quantiles

Standard deviation

Generate:

reports/drift_report.csv

reports/drift_report.json

No database.

36. VALIDATION GATES

After cleaning, run validation.

Check:

Row integrity

Column integrity

Data types

Missingness

Duplicates

ID uniqueness

Referential integrity

Business rules

Invalid values

Outlier behavior

Schema integrity

Result:

PASS

WARNING

FAIL

If a critical validation fails:

DO NOT PUBLISH FINAL DATASET

37. ROLLBACK

If a transformation causes quality degradation:

STOP

ROLL BACK

LOG FAILURE

DO NOT PUBLISH BAD OUTPUT

Example:

Before missingness: 5%

After: 12%

Validation:

FAIL

The original raw dataset must remain untouched.

38. POWER BI READINESS

Implement a dedicated Power BI validation layer.

Check:

Mixed data types

Date validity

Date consistency

Duplicate keys

Foreign keys

Relationship problems

Orphan records

High-cardinality fields

Dimension consistency

Numeric measures

Percentage representation

Currency fields

Identify:

Fact tables

Dimension tables

Primary keys

Foreign keys

Date dimension

Measures

Attributes

Provide a transparent:

Power BI Readiness Score

39. POWER BI RECOMMENDATIONS

Example:

POWER BI READINESS: 93/100

✓ Date fields valid

✓ Revenue numeric

✓ Customer_ID unique

✓ Region standardized

⚠ Transaction_ID is high cardinality

⚠ 2.4% orphan customer IDs

Recommendation:

Review customer relationship before creating the model.

Numbers must come from actual analysis.

40. DATA QUALITY SCORE

Calculate a transparent score based on relevant dimensions:

Completeness

Validity

Consistency

Uniqueness

Integrity

Schema quality

Power BI readiness

Example:

Before: 72/100

After: 94/100

Do not invent scores.

Document the methodology.

41. BEFORE/AFTER COMPARISON

Generate:

before_after_summary.json

before_after_summary.csv

Compare:

Rows

Columns

Missing values

Duplicate rows

Duplicate IDs

Invalid values

Invalid dates

Outliers

Quality score

Power BI readiness

42. DATA DICTIONARY

Generate:

data_dictionary.csv

Include:

column_name

original_name

data_type

description

nullable

unique_count

missing_count

missing_percentage

min

max

mean

median

example_values

cleaning_actions

lineage_id

43. PROFESSIONAL PDF REPORT

Generate:

reports/DataQX_Report.pdf

The report should contain:

Cover

DATAQX

DATA QUALITY & CLEANING REPORT

Include:

Project

Dataset

Date

Run ID

Executive Summary

Include:

- Dataset analyzed
- Project objective
- Initial quality
- Final quality
- Major issues
- Major fixes
- Remaining risks

Dataset Overview

Include:

Rows

Columns

File size

Data types

Data Quality Assessment

Include:

Missingness

Duplicates

Invalid values

Outliers

Schema issues

Consistency

Cleaning Actions

For each important transformation:

Problem

Detection

Action

Reason

Confidence

Impact

Data Lineage

Show:

Raw → Transformation → Clean

Data Drift

If historical profile files exist, show:

Previous

Current

Change

Severity

Validation Gates

Show:

PASS

WARNING

FAIL

Power BI Readiness

Include:

Data types

Dates

Keys

Relationships

Dimensions

High-cardinality warnings

Before vs After

Use tables/charts where useful.

Remaining Issues

Clearly list unresolved issues.

Recommendations

Provide actionable recommendations.

Technical Appendix

Include:

Libraries

Rules

Processing time

Memory usage

Input hash

Output hash

Software version

Configuration

Quality methodology

44. FRONTEND

The application should look like a professional analytics/data-quality SaaS product.

Use:

Sidebar

Navigation

Cards

Charts

Tables

Status badges

Progress indicators

Upload interface

Tabs

Filters

Tooltips

Loading states

Empty states

Error states

Avoid:

Plain HTML

Unstyled forms

Excessive animations

Huge text walls

Unnecessary colors

45. MAIN PAGES

Create:

1. Dashboard

2. Upload Dataset

3. Dataset Overview

4. Data Quality

5. Cleaning Actions

6. Before vs After

7. Data Lineage

8. Data Drift

9. Power BI Readiness

10. Data Dictionary

11. Reports & Downloads

Do NOT create a database-backed Run History page.

If previous run metadata files exist locally, provide a lightweight:

Run Files / Previous Reports

view based purely on filesystem metadata.

Do not create persistent cloud run history.

46. UPLOAD PAGE

Support:

Drag & Drop

Browse Files

Project Plan Upload

Display:

File name

File type

File size

Upload status

47. DATASET PREVIEW

Show:

Dataset name

File type

Rows

Columns

Size

Provide a preview table with:

Search

Sorting

Pagination

For large datasets, use sampled/limited preview.

Never send millions of rows to the browser.

48. DATA QUALITY DASHBOARD

Display:

Data Quality Score

Rows

Columns

Issues Detected

Issues Fixed

Review Required

Critical Issues

Power BI Readiness

Charts:

Missing values

Issue severity

Data type distribution

Before vs After

Problematic columns

Do not hardcode dashboard values.

All metrics must come from the backend.

49. ISSUE REVIEW

Show:

Issue

Column

Severity

Confidence

Affected records

Status

Recommended action

Filters:

Severity

Issue type

Column

Confidence

Status

Allow the user to review low-confidence transformations.

50. CLEANING WORKFLOW

Show:

Uploading



Profiling



Detecting Issues



Evaluating Confidence



Cleaning



Validating



Generating Reports



Complete

Provide meaningful progress messages.

51. USER CONTROL

User should be able to:

Analyze dataset

Review issues

View proposed transformations

Approve/reject review-required changes

Apply cleaning

Validate result

Download outputs

High-confidence transformations may be automatically applied.

Low-confidence transformations require review.

52. API ARCHITECTURE

Create clean FastAPI endpoints.

Possible endpoints:

POST /api/upload

POST /api/analyze

POST /api/clean

POST /api/validate

GET /api/profile/{run_id}

GET /api/issues/{run_id}

GET /api/lineage/{run_id}

GET /api/drift/{run_id}

GET /api/powerbi/{run_id}

GET /api/report/{run_id}

GET /api/download/{run_id}/{file}

These APIs must operate on filesystem-backed temporary run artifacts.

They must NOT perform database CRUD.

Do NOT create:

/api/users

/api/database

/api/db

/api/projects CRUD

unless explicitly required later.

53. API STATE MANAGEMENT

Do not create server-side persistent application state.

Use:

run_id

temporary directories

filesystem artifacts

request-scoped processing

where necessary.

If an API needs to retrieve results during an active run, locate them using the run's temporary/filesystem path.

Do not store state in a database.

54. FILE CLEANUP

Temporary processing files should be cleaned up safely when appropriate.

Implement a configurable cleanup policy.

For example:

data/temp/

can contain active processing artifacts.

Completed downloadable artifacts can remain for the current local workflow.

On cloud deployment, assume ephemeral lifecycle.

Do not build a permanent storage system.

55. SECURITY

Because users upload files:

Implement:

File type validation

Filename sanitization

Safe path handling

Maximum file size

Temporary directory isolation

No arbitrary code execution

No execution of uploaded files

Treat uploaded data as untrusted input.

56. ERROR HANDLING

Handle:

Empty files

Corrupt files

Unsupported files

Malformed CSV

Invalid Excel

Invalid JSON

Invalid Parquet

Wrong encoding

Missing headers

Duplicate headers

Memory problems

Invalid rows

Normal users should see friendly messages.

Technical stack traces belong in:

logs/errors.log

57. PERFORMANCE MONITORING

Measure:

Upload

File loading

Schema detection

Profiling

Issue detection

Cleaning

Validation

Drift detection

Power BI validation

Output generation

PDF generation

Generate:

logs/performance.log

and optionally:

performance_log.csv

58. BOTTLENECK DETECTION

Identify the most expensive processing stage.

Example:

Loading: 2.4s

Profiling: 4.1s

Duplicate detection: 8.3s

Cleaning: 3.1s

Validation: 1.9s

Report generation: 2.3s

The system must measure actual values.

59. LARGE DATASET HANDLING

Avoid:

$O(n^2)$

Unnecessary dataset copies

Repeated conversions

Unnecessary browser transfers

Prefer:

Vectorized operations

Chunk processing

Efficient dtypes

Sampling

Incremental profiling

Adapt processing to dataset size.

60. MULTI-FORMAT EDGE CASES

CSV

Handle:

Different delimiters

Quoted values

Embedded commas

Embedded newlines

Escaped quotes

BOM

Different encodings

Inconsistent row lengths

Excel

Handle:

Multiple sheets

Empty sheets

Hidden sheets

Header rows

Merged cells

Blank rows

Formula cells

Multiple tables

JSON

Handle:

Nested objects

Arrays

Missing fields

Inconsistent schemas

Parquet

Handle:

Schema metadata

Column types

Large files

61. TEST DATA

Create sample datasets with:

Missing values

Duplicates

Wrong types

Bad dates

Outliers

Invalid categories

Whitespace

Encoding issues

Invalid IDs

Negative values

Mixed formats

Referential integrity problems

Keep them separate:

data/samples/

62. TESTING

Implement:

Unit tests

For:

Detectors

Cleaners

Validators

Confidence engine

Lineage

Drift

Power BI rules

Integration tests

Test:

Upload → Analyze

Analyze → Clean

Clean → Validate

End-to-end

Test:

Upload

→ Analyze

→ Clean

→ Validate

→ Reports

→ Downloads

63. NO MOCKED SUCCESS

This is mandatory.

Do not claim:

50,000 rows processed

94/100 quality

323 tests passed

18.2 seconds runtime

unless those values came from actual execution.

Never fabricate results.

64. PHASED IMPLEMENTATION — ABSOLUTE REQUIREMENT

DO NOT IMPLEMENT ALL PHASES IN ONE PASS.

The development workflow MUST be:

PLAN



IMPLEMENT ONE PHASE



EXECUTE



TEST



INSPECT ACTUAL OUTPUT



IDENTIFY FAILURES



FIX



RETEST



REGRESSION TEST



VERIFY



MARK COMPLETE



NEXT PHASE

A feature is complete only when:

Code implemented

+

Tests written

+

Tests executed

+

Actual output inspected

+

Edge cases tested

+

Failures fixed

+

Regression tests passed

+

Documentation updated

Never mark a feature complete based on code existence alone.

65. IMPLEMENTATION PHASES

PHASE 0 — Environment & Architecture

Inspect:

Python

Node

npm

Existing project

Create architecture.

Verify:

python --version

node --version

npm --version

Do NOT implement the entire application.

PHASE 1 — FastAPI Foundation

Implement:

FastAPI

Health endpoint

Configuration

Logging

Filesystem utilities

Temporary run directory management

No database.

Verify:

GET /health

PHASE 2 — React Foundation

Implement:

React/Vite

Routing

Layout

Sidebar

Dashboard shell

Verify frontend actually runs.

PHASE 3 — Upload System

Implement:

Dataset upload

Project plan upload

File validation

Safe temporary storage

Test actual files.

PHASE 4 — Multi-Format Ingestion

Implement:

CSV

TSV

Excel

JSON

Parquet

Test each format.

PHASE 5 — Dataset Profiling

Implement complete profiling.

Compare actual results against known values.

PHASE 6 — Issue Detection

Implement:

Missing

Duplicates

Types

Dates

Categories

Outliers

Invalid values

Test intentionally broken datasets.

PHASE 7 — Confidence Engine

Implement:

HIGH

MEDIUM

LOW

Verify classification.

PHASE 8 — Cleaning Engine

Implement safe transformations.

Verify:

Raw unchanged

Clean output correct

Logs correct

PHASE 9 — Audit Logging

Implement file-based:

cleaning_log.csv

audit_log.csv

Verify transformations are recorded.

PHASE 10 — Data Lineage

Implement:

data_lineage.csv

data_lineage.json

Verify raw → transformation → output.

PHASE 11 — Validation Engine

Implement:

Schema validation

Business rules

Integrity

Quality validation

Test deliberately failing data.

PHASE 12 — Validation Gates & Rollback

Implement:

PASS

WARNING

FAIL

and rollback.

Test deliberately bad transformations.

PHASE 13 — Project Plan Integration

Implement:

project_plan.md

and project-aware cleaning.

Test the same dataset with different project plans.

PHASE 14 — File-Based Run Metadata

Implement run directories such as:

reports/runs/run_<id>/

Store metadata and artifacts only as files.

No database.

PHASE 15 — Data Drift

Implement:

Schema drift

Volume drift

Missingness drift

Category drift

Distribution drift

using historical profile files.

No database.

PHASE 16 — Power BI Validation

Implement:

Types

Dates

Keys

Relationships

Dimensions

High cardinality

Measures

Star-schema analysis

PHASE 17 — Data Dictionary & Summaries

Implement:

data_dictionary.csv

before_after_summary.csv

quality_report.json

validation_report.json

PHASE 18 — PDF Reporting

Generate:

DataQX_Report.pdf

Actually inspect the generated PDF.

PHASE 19 — Dashboard Integration

Connect frontend to real backend results.

No hardcoded metrics.

PHASE 20 — Lineage / Drift / Power BI UI

Connect all advanced analysis to the frontend.

PHASE 21 — Performance

Measure actual bottlenecks.

Optimize actual bottlenecks.

PHASE 22 — Security & Error Handling

Test malformed and unsafe input scenarios.

PHASE 23 — Full Testing

Run:

Unit

Integration

Frontend

End-to-end

Regression

Performance

PHASE 24 — Final End-to-End Validation

Run:

Upload

→ Project Plan

- Analyze
- Clean
- Validate
- Lineage
- Drift
- Power BI
- Reports
- Download

Verify every artifact.

66. AFTER EVERY PHASE

Report:

=====

PHASE <NUMBER>

=====

Implemented:

...

Tests executed:

...

Passed:

...

Failed:

...

Problems found:

...

Fixes:

...

Actual outputs inspected:

...

Regression:

PASS / FAIL

Status:

VERIFIED / NOT VERIFIED

=====

Do not continue when critical failures remain.

67. FINAL OUTPUT PACKAGE

The complete package should look like:

data/

└─ input/

└─ output/

└─ temp/

└─ samples/

data/output/

└─ cleaned_dataset.csv

└─ cleaned_dataset.xlsx

└─ data_dictionary.csv

reports/

└─ DataQX_Report.pdf

└─ quality_report.json

└─ before_after_summary.json

└─ validation_report.json

└─ drift_report.csv

└─ drift_report.json

└─ runs/

└─ history/

logs/

└─ audit_log.csv

└─ cleaning_log.csv

└─ data_lineage.csv

└─ data_lineage.json

└─ errors.log

└─ performance.log

No database files.

No database configuration.

No ORM.

68. FINAL DASHBOARD

The completed dashboard should show something similar to:

DATAQX

Dataset

Cybersecurity_Incidents.csv

Rows

50,000

Columns

24

Quality Score

95/100

Power BI Readiness

93/100

Issues Detected

17

Automatically Fixed

13

Review Required

4

Critical Issues

0

Validation

PASS

Data Drift

WARNING

Processing Time

18.42 sec

[View Issues]

[View Cleaning]

[View Lineage]

[View Drift]

[Power BI Readiness]

[Download Clean Dataset]

[Download PDF Report]

All values must come from actual processing.

69. FINAL DOWNLOAD CENTER

Provide:

[Download Clean Dataset]

[Download Data Dictionary]

[Download Cleaning Log]

- [Download Audit Log]
 - [Download Data Lineage]
 - [Download Drift Report]
 - [Download Validation Report]
 - [Download Before/After Summary]
 - [Download PDF Report]
-

70. FINAL QA CHECKLIST

Before claiming completion:

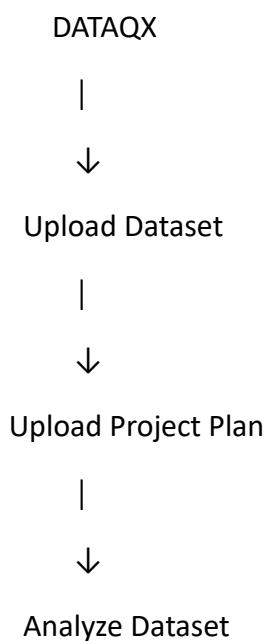
- ☐ No database
 - ☐ No PostgreSQL
 - ☐ No MySQL
 - ☐ No MongoDB
 - ☐ No SQLite
 - ☐ No Supabase database
 - ☐ No Neon database
 - ☐ No SQLAlchemy
 - ☐ No ORM
 - ☐ No database migrations
 - ☐ No database connections
 - ☐ No database CRUD
 - ☐ No persistent database storage
 - ☐ No database-backed run history
 - ☐ No database-backed audit logs
 - ☐ No database-backed lineage
 - ☐ No database-backed drift history
-
- ☐ No Docker

- ☐ Windows local execution works
- ☐ Frontend works
- ☐ Backend works
- ☐ Frontend connects to backend
- ☐ Upload works
- ☐ Project plan works
- ☐ CSV works
- ☐ TSV works
- ☐ Excel works
- ☐ JSON works
- ☐ Parquet works
- ☐ Profiling works
- ☐ Missing detection works
- ☐ Duplicate detection works
- ☐ Type detection works
- ☐ Numeric cleaning works
- ☐ Text cleaning works
- ☐ Category handling works
- ☐ Date validation works
- ☐ Outlier detection works
- ☐ Business rules work
- ☐ ID validation works
- ☐ Referential integrity works
- ☐ Confidence framework works
- ☐ Data lineage works
- ☐ File-based audit logs work
- ☐ File-based run metadata works
- ☐ File-based drift detection works

- [] Validation gates work
 - [] Rollback works
 - [] Project-specific rules work
 - [] Power BI validation works
 - [] Quality scoring works
 - [] Data dictionary works
 - [] PDF report works
 - [] Performance monitoring works
 - [] Large dataset handling works
 - [] Error handling works
 - [] Security basics work
 - [] Automated tests pass
 - [] Regression tests pass
 - [] End-to-end test passes
 - [] README complete
-

71. DEFINITION OF DONE

The project is complete only when the following real workflow works:



|



Dataset Profiling

|



Issue Detection

|



Confidence Engine

|



Safe Cleaning

|



Data Lineage

|



Validation Gates

|



Data Drift

|



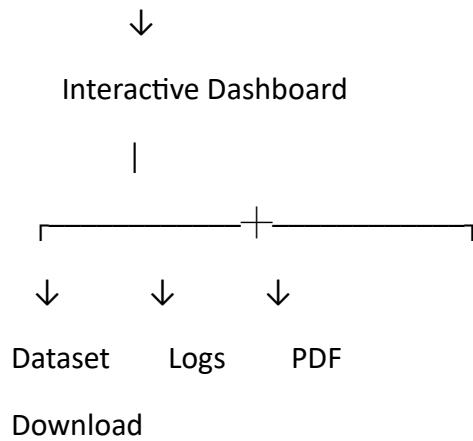
Power BI Readiness

|



Quality Score

|



Everything must be based on actual execution.

72. FINAL ARCHITECTURE CHECK

Before implementation begins, inspect the entire plan and verify:

DATABASE = NONE

PERSISTENT DATABASE = NONE

ORM = NONE

SQL = NONE

DATABASE AUTH = NONE

DATABASE CRUD = NONE

DATABASE MIGRATIONS = NONE

The only storage mechanism is:

FILESYSTEM

+

TEMPORARY PROCESSING DIRECTORIES

+

GENERATED REPORT/LOG FILES

Historical drift is based on:

previous profile/report JSON files

Audit logs are:

CSV/JSON/log files

Lineage is:

CSV/JSON files

Run metadata is:

JSON files

There must be no contradictory database requirement anywhere in the implementation.

73. FINAL PRODUCT PHILOSOPHY

DataQX should NOT be presented as:

"A Python script that cleans datasets."

It should be presented as:

"A stateless, project-aware Data Quality & Preparation Platform that automatically profiles, detects, cleans, validates, audits, tracks lineage, detects data drift, evaluates Power BI readiness, and generates professional analytics-ready datasets and reports."

The platform should be capable of handling:

Sales

Finance

E-commerce

HR

Cybersecurity

Operations

IoT

Customer Analytics

Marketing

Power BI

without rewriting the core engine.

74. START HERE

DO NOT implement the entire system immediately.

Start with:

PHASE 0 ONLY

You must:

1. Inspect the existing project directory.
2. Inspect Python version.
3. Inspect Node.js version.
4. Inspect npm version.
5. Determine installed dependencies.
6. Check whether any existing files conflict with the architecture.
7. Create the database-free architecture.
8. Create the initial README.
9. Create .gitignore.
10. Create minimal backend/frontend skeletons only where appropriate.
11. Verify the environment.
12. Confirm that NO database dependency exists.

Then actually execute the Phase 0 checks.

Report:

PHASE 0 STATUS

Environment:

...

Architecture:

...

Files created:

...

Commands executed:

...

Actual results:

...

Database dependencies:

NONE

Problems:

...

Fixes:

...

Verification:

PASS / FAIL

Do NOT begin Phase 1 until Phase 0 is actually verified.

FINAL COMMANDMENT

IMPLEMENT → EXECUTE → INSPECT → TEST → FIX → RETEST → VERIFY → CONTINUE

Never skip this cycle.

Never fabricate results.

Never mark a feature complete based on code existence alone.

Never overwrite raw data.

Never silently modify uncertain data.

Never publish a dataset that fails a critical validation gate.

Never introduce a database.

Never introduce Docker.

Build DataQX as a **simple, deployable, reliable, stateless, portfolio-grade data quality platform**.

The final deployment target is:

React + Vite



Vercel

FastAPI + Python DataQX Engine



Render

Filesystem / Temporary Processing



Generated Reports & Downloads

NO DATABASE

Start with **PHASE 0 ONLY**.